Spreading Factor and RSSI for Localization in LoRa Networks: A Deep Reinforcement Learning Approach

Yaya Etiabi[1], Mohammed Jouhari[1], and El Mehdi Amhoud[1]

[1]School of Computer Science, Mohammed VI Polytechnic University, Benguerir, Morocco
Email: {yaya.etiabi, mohammed.jouhari, elmehdi.amhoud}@um6p.ma

arXiv:2205.11428v1 [eess.SP] 23 May 2022

***Abstract*—Recent advancements in Internet of Things (IoT) technologies have resulted in a tightening of requirements from various applications including localization in LoRa networks. To address the growing demand for LoRaWAN-powered IoT location-based services, accurate localization solutions are more crucial than ever. As such, in this work, we develop an accurate deep neural network based localization framework over a LoRa network by proposing a novel approach that builds the network radio map with the combination of RSSI recordings and the spreading factors (SF) used by LoRa devices during the transmissions. Then, we validate our framework using a publicly available experimental dataset recorded in an urban LoRa network. The performance evaluation shows the prominence of adding the spreading factor as an additional fingerprint, since we can achieve, by our approach, an improvement in localization accuracy by up to 6.67% compared to the state-of-the-art methods which employ uniquely the RSSI fingerprints. Additionally, we provide an analysis of the impact of the SF on the localization performance which reveals that the localization accuracy relies on the SF used for position request. Finally, we propose a deep reinforcement learning based localization system to capture the ever-growing complexity of LoRa networks environment and cope with the scalability issue in LoRa enabled massive IoT, and the results show an improvement of 63.3% in terms of accuracy.**

***Index Terms*—LoRaWAN, Localization, Massive IoT, RSSI Fingerprinting, Spreading Factor, Deep reinforcement learning**

I. Introduction

The Internet of Things (IoT) is continuously reshaping the way we interact with our environment. This is done thanks to the widespread of cutting edge sensing devices deployment at an unprecedented pace, and the development of low cost and long range communication technologies such as NB-IoT, SigFox, LTE-M, and LoRaWAN. The latter is the most widely used IoT communication technology and seems very promising for IoT applications involving limited resources, regarding its low cost, long range communication capability, and its ease of deployment as enlightened in [1]. With the growing use of smart objects, we are living in the beginning of a new IoT age, namely massive IoT (MIoT), a new design paradigm that envisions networks composed of billions to trillions of tiny sensors connecting with one another to provide creative real-time solutions. Thus, one of the most important services for this new IoT paradigm is accurate location-based services.

Since IoT networks are growing, GPS-based solutions can no longer support the location services either due to environmental challenges or due to the cost of their deployment. As such, many research works have attempted to provide LoRa powered localization systems by exploiting the LoRa networks properties such as fingerprinting, time of arrival (ToA), angle of arrival (AoA) and time difference of arrival (TDoA) [2], [3]. More recently, many machine learning algorithms have been proposed to enhance the localization accuracy and to reduce the online prediction complexity involved in the traditional methods [4]–[7]. These machine learning techniques are mostly based on the fingerprints prerecorded in the network, which represent the radio map of the network.

Nonetheless, unlike WiFi [8], in LoRa communication, such RSSI fingerprints may lack representativeness of the network radio map since the RSSI fingerprinting of the covered area is affected by the spreading factor (SF) which is a LoRa key parameter. Indeed the SF, an important metric in LoRaWAN communication, controls the number of chirps (data carriers) transmitted each second. With a higher SF, there are fewer chirps per second, and hence less data is processed per second. When transmitting the same amount of data with a greater SF, more airtime is required. To achieve more airtime, the LoRa node operates and runs for longer periods of time, costing more energy. As a result, SF assignment is a performance-versus-energy-consumption trade-off that is critical for LoRa-based IoT applications. The network determines the SF (graded 7-12) based on the environmental conditions between the communication device and the gateway in order to fully use the benefits of LoRa technology and optimize its performance. Consequently, many research works have been proposed for optimal SF allocation [9].

In this work, we propose a new localization framework that takes into consideration the SF in the construction of the RSSI radio map. Indeed, the objective of our study is to design a robust localization system for end-devices self-localization in constrained and GPS- challenged outdoor environments in a LoRa network, in a context of MIoT. The motivation of using SF to construct the radio map of the area of interest is that different SFs lead to different RSSI fingerprints at a

given location. In other words, to each SF corresponds a radio map. Thus, taking into account the SF will provide a fully comprehensive radio map that is expected to present better localization performance. Such RSSI fingerprints construction may lead to high complex datasets and the nature of the wireless environment may greatly affect the quality of RSSI signals which in turn may mislead the deep neural network (DNN) based model, leading to a performance degradation of the overall localization system. Consequently, we propose a new formulation of the localization problem with a deep reinforcement learning (DRL) approach which is expected to undertake the complex nature of the IoT environment and achieve better localization accuracy. The contribution of our work is as follows:

- First of all, we show the prominence of using the SF as an additional feature in the building of the RSSI fingerprints.
- Afterwards, we investigate the impact of the SF on the localization accuracy by assessing the localization system accuracy with respect to the SF allocation policy. This enables a better trade-off between network performance and localization accuracy.
- Finally, we propose the use of DRL approach to capture the environmental complexity and the scalability issues induced by the MIoT networks. This has been done using the *OpenAI Gym* environment and the *keras-rl2* library.

The reminder of this paper is organized as follows. Section II describes the the proposed localization system model, while Section III introduces our DRL approach. In Section IV, we present the performance evaluation of the two localization systems. Finally in Section V, we conclude and set forth our perspectives.

II. DNN BASED LOCALIZATION FRAMEWORK

As stated earlier, the goal of this work is to develop a robust localization system in LoRa networks that will enable IoT devices to self-localize in outdoor environments with harsh wireless communication conditions where GPS fails to perform efficiently. To do so, we start by proposing a combined SF and RSSI fingerprinting approach to construct our dataset.

A. RSSI data acquisition

We consider a dense LoRa network composed of N LoRa end-devices so called nodes, deployed in an area of dimension $L \times W$, and M LoRa gateways distributed across the network.

In LoRa as well as in other wireless networks, the RSSI is strongly related to the path loss experienced in the studied environment. Basically, the path loss is computed as follows:

$$PL = P_{T_x} - P_{R_x}, \tag{1}$$

where P_{T_x} and P_{R_x} are respectively the transmit and received power. Using the log-distance path loss model as in [10], PL is expressed by the following equation :

$$PL = PL_0 + 10\beta \log_{10} \frac{d}{d_0} + X_\sigma, \tag{2}$$

where β is the path loss exponent, d_0 is the reference distance, X_σ represents the log-normal shadowing with standard deviation σ in dB and PL_0 the path loss at the reference distance d_0. PL_0 given by the free space propagation model is defined as:

$$PL_0 = 10 \log_{10} \left(\left(\frac{4\pi d_0}{\lambda} \right)^2 \right), \tag{3}$$

with λ the wavelength of the signal.

The RSSI which is the received power P_{R_x} is then computed as follows:

$$RSSI = P_{R_x} = P_{T_x} - \left[PL_0 + 10\beta \log_{10} \frac{d}{d_0} + X_\sigma \right], \tag{4}$$

meaning that the RSSI value captured from the mth gateway at the nth position at time t can be expressed as:

$$\begin{aligned} RSSI_{n,t}^m = P_{Tx} - \Bigg[& 20 \log_{10} \left(\frac{4\pi d_0}{c} \right) + 20 \log_{10}(f) + \\ & 10\beta \log_{10} \left(\frac{d_n^m}{d_0} \right) + X_{\sigma,n,m}^t \Bigg]. \end{aligned} \tag{5}$$

Although LoRa has a good sensitivity, the RSSI can be far below the noise floor causing the problem of proper decoding of the signal. In such case, no RSSI value is recorded. Thus, this measurement point is considered as a missing value which is handled in the data preprocessing step described in Section II-B. The same treatment is applied when the gateway is out-of-range or no longer available. The recording of RSSI with respect to the receiver sensitivity is described by the following equation :

$$RSSI_{n,t}^m = \begin{cases} RSSI_{n,t}^m & \text{if } RSSI_{n,t}^m > S_{n,t}^m \\ NaN & \text{Otherwise} \end{cases}, \tag{6}$$

where S, given by Eq. (7), is the sensitivity which depends on the received SNR threshold which in turn is related to the spreading factor SF.

$$S = -174 + 10 \log_{10} \mathrm{BW} + \mathrm{NF} + \mathrm{SNR}, \tag{7}$$

where BW is the bandwidth in KHz, NF the noise figure of the receiver in dB, and SNR the signal to noise ratio. Thus, as depicted in Fig. 1, the minimum recorded RSSI value depends on the spreading factor allocated to the node, given a specific bandwidth. Overall, the higher the SF the higher the sensitivity and by extent the longer the communication range and the better the coverage.

B. RSSI data preprocessing

As seen in the previous section, the constructed RSSI database contains missing values that need to be handled. To do so, we consider the RSSI thresholds for different SF values so that a missing value is replaced by the RSSI threshold of the SF allocated during the communication round. This is indeed described by :

$$RSSI_{n,t}^m = \begin{cases} S_{n,t}^m(SF) & \text{if } RSSI_{n,t}^m = NaN \\ RSSI_{n,t}^m & \text{Otherwise} \end{cases}. \tag{8}$$

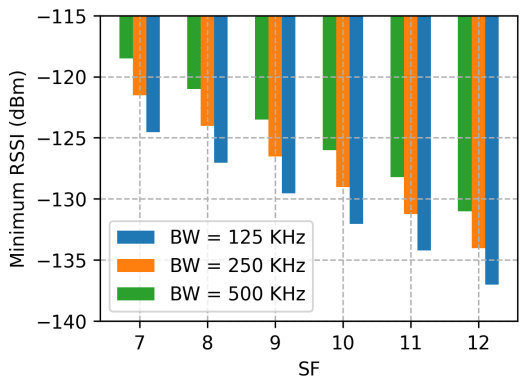

Fig. 1: Minimum RSSI Vs SF

TABLE I: Parameters of the DNN model

Parameter	Description	Value
Optimizer	DNN Model Optimizer	Adam
μ	Learning rate	0.0005
β_1, β_2	Exponential decay rates	$0.1, 0.99$
N_i	Input layers units	10
N_h	Hidden layer units	$512 \times 256 \times 128 \times 64 \times 32$
D_1, D_2, D_3	Dropout layers	$0.3, 0.2, 0.1$
N_o	Output layer units	2
$\sigma_h(\cdot)$	Hidden Activation function	ReLu
$\sigma_o(\cdot)$	Output Activation function	Linear
b	Batch size	512

Afterwards, the SF is added to the RSSI vector as an additional feature. Finally, since we are using gradient-based optimization methods, the RSSI values are normalized using the min-max normalization given by Eq. (9), in order to speed-up the learning process and to reduce the risk of algorithm divergence.

$$RSSI_i = \frac{RSSI_i - \min(RSSI)}{\max(RSSI) - \min(RSSI)}. \tag{9}$$

C. DNN model training

In this subsection, we define a DNN architecture whose configuration is shown in TABLE I. The training of the model can be seen as a minimization problem considering the loss function below :

$$\min_{\boldsymbol{W}} \frac{1}{N} \sum_{n=1}^{N} f\left(\boldsymbol{W}, [\boldsymbol{RSSI}_n \ SF_n], \boldsymbol{C}_n\right), \tag{10}$$

where $\boldsymbol{C}_n$ is the nth reference position coordinates corresponding to the captured $\boldsymbol{RSSI}_n$ at that location, and $\boldsymbol{W}$ represents the weights or parameters of the DNN model. The optimal weights $\boldsymbol{W}^*$ are obtained through the minimization of the mean absolute error loss function associated with the DNN model.

III. Localization with Reinforcement Learning

Reinforcement Learning (RL) is "trial and error process" [11] that can be seen as a general system configuration in which an agent interacts with its environment in an action-reward manner. Indeed, the RL agent takes observations from the environment, then performs specific actions and gets rewarded from the environment regarding the performed actions. As a consequence, the agent learns from its past errors to improve its decision making efficiency, while trying new behaviours that can potentially increase its reward in the long-run as well as the intermediate rewards. The ultimate goal of the RL agent is thus to learn an optimal, or nearly-optimal, policy that maximizes the reward function or other user-provided reinforcement signal that accumulates from the immediate rewards. Such optimization problem can be addressed using Markov Decision Process (MDP) [11]. Hence, we model our localization problem as a MDP in the next subsection.

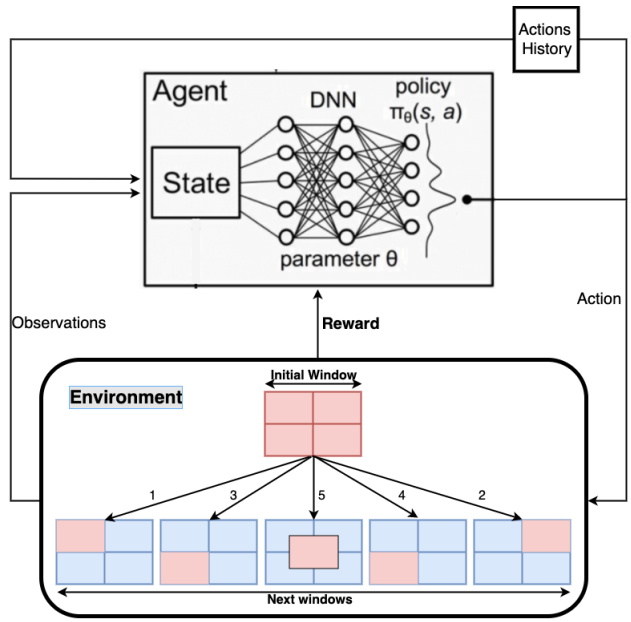

Fig. 2: Deep Reinforcement Learning based Localization Framework

A. Localization as a Markov Decision Process

We consider the localization system as a RL agent whose goal is to accurately localize IoT devices in the LoRa network by interacting with the wireless environment. Our considered environment is defined by the RSSI signals and the geometry of the LoRa network, within which the agent shifts and changes a bounding square window (w) through a sequence of actions and goes to the next state after performing a specified action under the current observation. When the targeted IoT device enters the environment and gets an RSSI signal, the localization agent is required to gradually locate it by enclosing it in a small enough window. The agent should identify how to move and rearrange the window at each phase of the localization process in order to efficiently localize the target in few steps, as shown in Fig. 2. Therefore, we formulate the localization problem as a dynamic Markov decision process characterized by the following components:

1) State Space: The state space of the localization problem encompasses the RSSI measurements $\mathbf{RSSI}$, the spreading factors SF_s used during measurements, the search window $\mathbf{\Omega}$ characterized by the center coordinates of the localization agent positions $\mathbf{\Theta}$ and the window size d, and the history vector $\mathbf{H}$ of actions taken so far.

At at given time step t, the state of the localization agent is defined by $s_t = [\mathbf{RSSI}_t, SF_t, \mathbf{\Omega}_t, \mathbf{H}_t]$ where $\mathbf{\Omega}_t = [\mathbf{\Theta}_t, d_t]$ with $\mathbf{\Theta_t} = (x_t,\ y_t)$.

The actions below mentioned, are encoded by their numerical values ranging from 1 to 5 as shown in Fig 2. Thus in order to avoid a misinterpretation of the history vector by the Q-Network (which will be discussed later), the actions are one-hot encoded in the history vector, i.e, if we save the last n actions in the actions history vector, the ith element is a 5-sized binary vector with all values set to 0 except at the position equal to the numerical value of the corresponding action, which is set to 1.

2) Action Space: The action space represents the set of actions the agent can perform when trying to locate a target. In our formulation, we distinguish 5 possible actions depicted in Fig. 2 and defined as follows:
- UP-LEFT: $\mathbf{\Theta}_{t+1} = (x_{t+1}, y_{t+1}) = (x_t - \mathrm{d}_t/2, y_t + \mathrm{d}_t/2)$
- UP-RIGHT: $\mathbf{\Theta}_{t+1} = (x_{t+1}, y_{t+1}) = (x_t + d_t/2, y_t + d_t/2)$
- DOWN-LEFT:
$\mathbf{\Theta}_{t+1} = (x_{t+1}, y_{t+1}) = (x_t - \mathrm{d}_t/2, y_t - \mathrm{d}_t/2)$
- DOWN-RIGHT:
$\mathbf{\Theta}_{t+1} = (x_{t+1}, y_{t+1}) = (x_t + \mathrm{d}_t/2, y_t - \mathrm{d}_t/2)$
- CENTER: $\mathbf{\Theta}_{t+1} = (x_{t+1}, y_{t+1}) = (x_t, y_t)$

For each of the above actions, the search window is shrunked as follows: $\mathrm{d}_{t+1} = \alpha \times d_t$, with $\alpha = 0.5$ in our formulation.

3) Reward Function: Our reward function is based on the normalized intersection of windows noted $\mathrm{IoW}(\mathbf{\Omega}, \mathbf{\Omega}^*) \in [0,\ 1]$ where $\mathbf{\Omega}$ is the search window and $\mathbf{\Omega}^*$ the target window. $\mathrm{IoW}(\mathbf{\Omega}, \mathbf{\Omega}^*)$ represents the overlapping area of the two windows normalized by the area of the search window and defined as:

$$\mathrm{IoW}(\mathbf{\Omega}, \mathbf{\Omega}^*) = \mathrm{area}(\mathbf{\Omega} \cap \mathbf{\Omega}^*) / \mathrm{area}(\mathbf{\Omega}). \tag{11}$$

The reward is then defined as:

$$r_{a_t}(s_t, s_{t+1}) = \begin{cases} +\xi & \text{if } \mathrm{IoW}(\mathbf{\Omega}^{s_{t+1}}, \mathbf{\Omega}^*) \in [\Delta, 1] \\ +\phi & \text{if } \mathrm{IoW}(\mathbf{\Omega}^{s_{t+1}}, \mathbf{\Omega}^*) \in [\mathrm{IoW}(\mathbf{\Omega}^{s_t}, \mathbf{\Omega}^*), \Delta[\\ -\xi & \text{otherwise} \end{cases}, \tag{12}$$

where Δ is the the overlapping threshold of the search and the target windows, ξ is the stopping reward and ϕ is the reward for each good action that takes the localization agent closer to the target. In our simulations, these paramters have been set to 0.5, 10 and 1 respectively.

B. Deep Q-Network based localization

The concept of DQN inroduced by DeepMind in [12] emanates from the aim of combinining RL and DNNs at scale to solve complex problems at superhuman level. Basically, DQN was developed by enhancing the classic RL algorithm called Q-Learning with DNN and the technique known as experience replay. Indeed, Q-learning as its name suggests is based on the notion of a Q-function most often represented by the so-called Q-table that maps the expected return or discounted sum of rewards $Q^\pi(s, a)$ obtained from state s, by taking action a and then following a policy π. Then, the optimal Q-function $Q^*(s, a)$ obeys the Bellman optimality equation given by:

$$Q^*(s, a) = \mathbb{E}\left[r + \gamma \max_{a'} Q^*(s', a')\right]. \tag{13}$$

Q-Learning intends to estimate the Q-function by using the Bellman Equation (13) as an iterative update formulated as:

$$Q_{i+1}(s, a) \leftarrow \mathbb{E}\left[r + \gamma \max_{a'} Q_i(s', a')\right]. \tag{14}$$

Such iterative formulation converges to the optimal Q-function $Q_i \rightarrow Q^*$ as $i \rightarrow \infty$ as shown in [13]. However, in localization problem, it is not possible to represent the Q-function as a table with values for each combination given the nature of the localization task where the state space is continuous. Instead, we define a neural network based approximation function with parameter θ, known as the DQN such that $Q(s, a; \theta) \approx Q^*(s, a)$. To do so, the DQN is trained by iteratively adjusting the parameter θ_i to reduce the loss function given by :

$$L_i(\theta_i) = \mathbb{E}_{s,a,r,s' \sim \rho(.)}\left[(y_i - Q(s, a; \theta_i))^2\right], \tag{15}$$

where $y_i = r + \gamma \max_{a'} Q(s', a'; \theta_{i-1})$ and ρ the distribution over transitions $\{s, a, r, s'\}$ collected from the environment.

During the training, ρ follows the ϵ-greedy policy which chooses between the greedy action (exploitation) with probability ϵ and a random action (exploration) with probability $1-\epsilon$ to ensure good coverage of the state-action space. Moreover, at each training iteration, we decay the ϵ value so that the exploitation probability increases as the agent is learning.

In standard Q-learning, the loss function is computed using only the last transition $\{s, a, r, s'\}$. In our training, we use the *Experience Replay*, a technique introduced in [13] to make the network updates more stable. Therefore, at each time step, the transition vector $\{s, a, r, s'\}$ is stored in a circular buffer named the *replay buffer*. Then, we use mini batches from samples in the replay buffer to update the Q-network.

IV. Performance Evaluation and Discussion

A. Simulation results

For the simulation, we consider a squared urban area of size $5 \times 5\ km^2$ covered by a LoRa powered IoT network equipped with M = 10 GWs. According to the experimental measurements done in [14], the environment variables PL_0, β and σ are set to 128.95, 2.32 and 7.8 correspondingly. We consider 10000 reference positions randomly scattered in this 25 km^2 network with the number of repetitions T set to 10. The RSSI values are created using Eq. (5) with a random

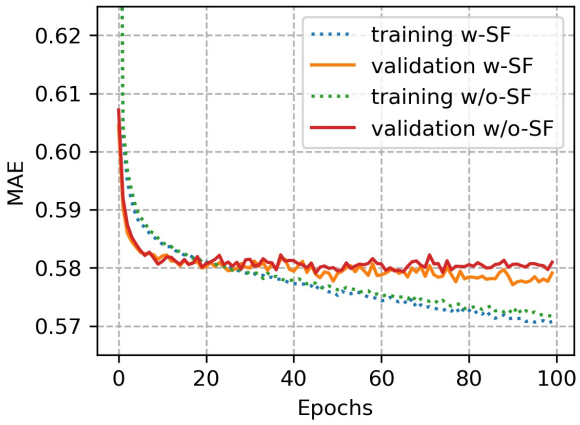

Fig. 3: Model Evaluation using simulation data

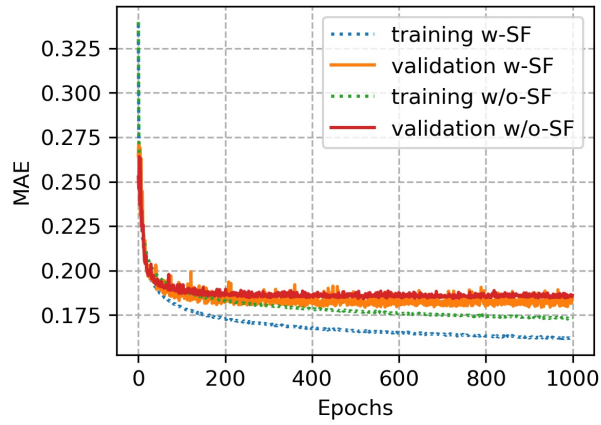

Fig. 5: DNN model evaluation with and without SF

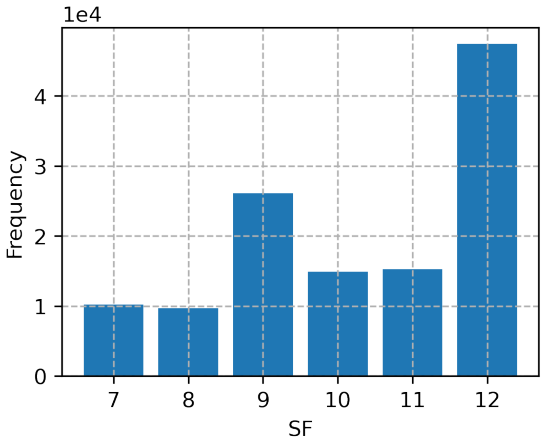

Fig. 4: SF distribution in the dataset

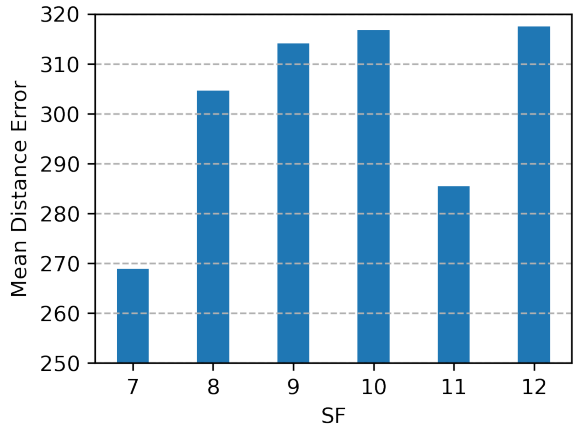

Fig. 6: Evaluation of the SF impact on the localization accuracy

seed of 200 to make the simulated RSSI dataset reproducible. Then we configure a DNN model according to TABLE I and perform the training whose results are depicted in Fig 3. This result confirms the prominence of using SF in the radio map construction since it improves the training as well as the validation accuracy of the proposed DNN model.

B. Experimental validation

In this section, we use a publicly available experimental dataset to assess the performance of our model. Indeed, we use the LoRaWAN dataset published by Aernouts et al. in [15], which was gathered over a 52 km^2 area in Antwerp, Belgium, by attaching LoRa modules to postal service vehicles and sending a message every minute to 68 LoRaWAN gateways. The experiment has been conducted for three months, resulting in 123,528 RSSI samples with the associated SFs used during the transmissions.

In Fig. 4, we explore the SF values jointly recorded during the RSSI data collection to ensure that our method can be adopted. Overall, we can see that all the SF values are well represented with a dominance of SF12.

TABLE II: Mean distance error (m) with and without SF

Model	Training	Validation	Baseline
w/o-SF	319.84	365.03	398.40
w-SF	**298.51**	**359.71**	-

Afterwards, we set the input of the DNN model described in TABLE I to $N_i = 69$(68 RSSI values from the 68 gateways + the transmission SF). The results are presented in Fig. 5 which reveals that with the SF as an additional feature input to the model, the convergence speeds up. The accuracy is improved by up to 6.67% as shown in TABLE II where the evaluation is done using the mean distance error metric.

Moreover, we evaluate the localization performance with respect to the SF feature and the result is presented in Fig. 6. We observe in general that the SF possesses an effect on the localization accuracy. Indeed, the best localization accuracy is obtained for SF7 followed by SF11 while SF12 leads to the worst localization performance after SF8, SF9 and SF10 respectively. More specifically, compared to SF12, using SF7 can improve the localization accuracy by $\sim 18\%$.

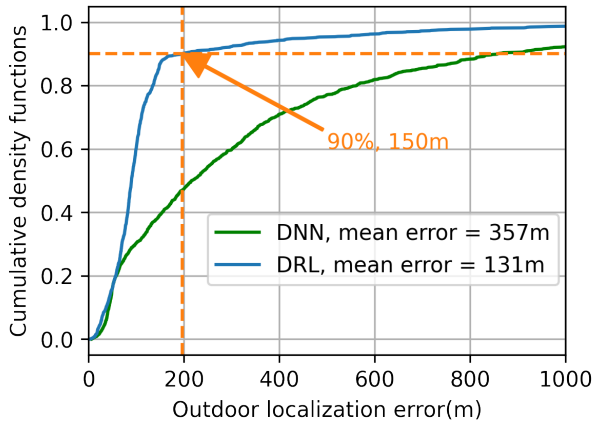

Fig. 7: Evaluation of DRL-based Localization system

C. Performance evaluation of the DRL system

Since our DRL-based localization is a top-down process, we start by defining an initial search widow ($\mathbf{\Omega}_0$) representing the smallest square that contains the whole LoRa network, and the target window ($\mathbf{\Omega}^*$) which determines how accurate we want our model to be. Using the experimental LoRa RSSI dataset of [15], each sample corresponds to an episode for our localization agent and its target window is defined by the corresponding label as $\mathbf{\Omega}^* = (\mathbf{\Theta}^* = (x^*, y^*), d^*)$, where x^*, y^* represent respectively the longitude and the latitude of the RSSI sample. d^* is the half length of the squared window centered at $\mathbf{\Theta}^*$ and represents the chosen precision for our system. The initial search window is characterized by its center $\mathbf{\Theta}_0$ and its half length d_0 defined respectively by $\left(\frac{x_{min}+x_{max}}{2}, \frac{y_{min}+y_{max}}{2}\right)$ and $\frac{max(x_{max}-x_{min}, y_{max}-y_{min})}{2} + d^*$, where d^* is set to 2.5% regarding the network geometry. We configure the Q-network with $128 \times 128 \times 64$ hidden layers, a learning rate of 10^{-4}, a batch size of 64 and a replay memory of length 5000. The remaining parameters are set as follows: the ϵ-greedy policy $\epsilon = 1$ (decaying at each iteration until $\epsilon_{min} = 0.001$, the discount factor $\gamma = 0.1$, and the actions history vector size $|H| = 10$. We run the simulation for iterations so-called epochs where each iteration represents a walk through the entire dataset. For time constraints, we only choose a subset of the dataset (1000 samples). Using the *OpenAI Gym* environment to model ours and the *keras-rl2* library to implement our DQN, we run the simulation for 200 iterations whose results are depicted in Fig. 7. It follows that the DRL-based approach substantially improves the localization accuracy with an increase of 63.3% compared to the DNN-based method earlier presented in this work. Moreover, it is worth noting that the DRL model has been trained only on a small subset of the dataset whereas the DNN used the entire dataset, which make the DRL approach a very promising method for future LoRa localization.

V. Conclusion

In this work, we have proposed a novel deep neural network-based localization framework for LoRa-powered IoT networks. Our method is the first to combine the spreading factor with the RSSI measurements in the construction of the LoRa network radio map. The evaluation results show that our method can improve the localization accuracy by up to 6.67% compared to state-of-the-art methods and highlight the correlation between the SF allocation and the localization system performance. Moreover, we proposed a deep reinforcement learning-based localization system as a response to the performance requirement dictated by the context of MIoT, and it showed good results with an improvement of 63.3% in the accuracy at an earlier training stage. To further improve our results, in addition to more robust RSSI data preprocessing technique and an hyper-parameter tuning, a joint optimal SF allocation and high accuracy localization method will be investigated in future work.

References

[1] M. Jouhari, E.-M. Amhoud, N. Saeed, and M.-S. Alouini, "A survey on scalable LoRaWAN for Massive IoT: Recent advances, potentials, and challenges," *ArXiv*, vol. abs/2202.11082, 2022.

[2] C. Gu, L. Jiang, and R. Tan, "LoRa-based localization: Opportunities and challenges," in *EWSN*, 2019.

[3] Y. Etiabi, E. M. Amhoud, and E. Sabir, "A distributed and collaborative localization algorithm for internet of things environments," in *Proceedings of the 18th International Conference on Advances in Mobile Computing & Multimedia*, 2020, pp. 114–118.

[4] I. B. F. de Almeida, M. Chafii, A. Nimr, and G. Fettweis, "Blind transmitter localization in wireless sensor networks: A deep learning approach," *32nd Annual International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC)*, pp. 1241–1247, 2021.

[5] T. Janssen, R. Berkvens, and M. Weyn, "Comparing machine learning algorithms for RSS-based localization in LPWAN," in *3PGCIC*, 2019.

[6] C. J. Bouras, A. Gkamas, S. A. K. Salgado, and N. Papachristos, "A comparative study of machine learning models for spreading factor selection in LoRa networks," *Int. J. Wirel. Networks Broadband Technol.*, vol. 10, pp. 100–121, 2021.

[7] W. Njima, M. Chafii, A. Nimr, and G. P. Fettweis, "Deep learning based data recovery for localization," *IEEE Access*, vol. 8, pp. 175 741–175 752, 2020.

[8] Y. Etiabi, E. Amhoud, and E. Sabir, "Huber estimator and statistical bootstrap based light-weight localization for IoT systems," *7th International Symposium on Ubiquitous Networking, UNet 2021*, vol. 12845, pp. 79–92, 2020.

[9] F. Benkhelifa, Z. Qin, and J. McCann, "Minimum throughput maximization in LoRa networks powered by ambient energy harvesting," in *IEEE International Conference on Communications (ICC)*, 2019, pp. 1–7.

[10] W. Njima, M. Chafii, A. Nimr, and G. Fettweis, "Convolutional neural networks based denoising for indoor localization," *93rd Vehicular Technology Conference (VTC2021-Spring)*, pp. 1–6, 2021.

[11] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. The MIT Press, 2018.

[12] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. A. Riedmiller, A. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[13] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller, "Playing Atari with deep reinforcement learning," *ArXiv*, vol. abs/1312.5602, 2013.

[14] J. Petajajarvi, K. Mikhaylov, A. Roivainen, T. Hanninen, and M. Pettissalo, "On the coverage of LPWANs: range evaluation and channel attenuation model for LoRa technology," in *International Conference on ITS Telecommunications (ITST)*, 2015, pp. 55–59.

[15] M. Aernouts, R. Berkvens, K. Van Vlaenderen, and M. Weyn, "Sigfox and LoRaWAN datasets for fingerprint localization in large urban and rural areas," *MDPI Data*, vol. 3, no. 2, p. 13, 2018.